API REFERENCE

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

# SDDC Manager API Handbook

Complete API reference for SDDC Manager including authentication, lifecycle operations, and automation workflows.

REST API | AUTHENTICATION | LIFECYCLE | AUTOMATION

**VCF 9.0** VMware Cloud Foundation

© 2026 Virtual Control LLC — All Rights Reserved

PROPRIETARY & CONFIDENTIAL

# VCF 9.0.1 — SDDC Manager REST API Handbook

**Environment:** VCF 9.0.1 Nested Lab — Dell Precision 7920 **SDDC Manager:** `https://sddc-manager.lab.local` (192.168.1.241) **API Base Path:** `/v1/` **Authentication:** Bearer Token (JWT) **Date:** February 2026

---

# Table of Contents

1. API Architecture Overview
2. Authentication & Token Management
3. System Health & Information
4. Credentials Management
5. NSX-T Cluster Inventory
6. Task Lifecycle & Monitoring
7. Resource Locks & Concurrency
8. Hosts Management
9. Domains & Clusters
10. Bundles & Lifecycle Updates
11. Notifications & Events
12. VCF Services & Depot
13. Complete Endpoint Reference
14. Python Automation Scripts
15. Postman Collection
16. Troubleshooting API Issues
17. Lab-Tested Workflows
18. References

---

# 1. API Architecture Overview

## How the SDDC Manager API Works

SDDC Manager exposes a REST API under the `/v1/` base path. All operations require Bearer token authentication obtained from the `/v1/tokens` endpoint. The API is primarily **asynchronous** — most mutating operations (upgrades, deployments, rotations) return a task ID that must be polled for completion.

```
+-----------------------------+
|    VMware Cloud Foundation   |
|        (VCF Platform)        |
+---------------+-------------+
                |
          Exposes REST API
                |
                v
+-----------------------------+
|      SDDC Manager API        |
|     Base Path: /v1/          |
+---------------+-------------+
                |
    +-----------+-----------+-----------+-----------+
    |           |           |           |           |
    v           v           v           v           v
+--------+ +----------+ +--------+ +----------+ +--------+
|  Auth  | |  Infra   | |  Tasks | |  Locks   | | Creds  |
| /v1/   | | /v1/     | | /v1/   | | /v1/     | | /v1/   |
| tokens | | hosts    | | tasks  | | resource | | creds  |
|        | | domains  | | {id}   | | -locks   | |        |
+--------+ | clusters | +--------+ +----------+ +--------+
           | nsxt-    |
           | clusters |
           +----------+
```

## API Layers

| Layer | Endpoints | Purpose |
|---|---|---|
| **Authentication** | `/v1/tokens` | Bearer token issuance and refresh |
| **Infrastructure** | `/v1/hosts`, `/v1/domains`, `/v1/clusters`, `/v1/nsxt-clusters` | Inventory and health of SDDC components |

| | | |
|---|---|---|
| **Workflow** | `/v1/tasks/{id}` | Status tracking for all async operations |
| **Concurrency** | `/v1/resource-locks` | Prevents conflicting operations on shared resources |
| **Credentials** | `/v1/credentials` | Secure storage and lifecycle of platform passwords |
| **Lifecycle** | `/v1/bundles`, `/v1/upgrades`, `/v1/upgradables` | Bundle management and component upgrades |
| **System** | `/v1/system`, `/v1/system/health`, `/v1/system/notifications` | Health checks, notifications, and system info |

## Key Concepts

- **Bearer Token Auth** — Every call requires `Authorization: Bearer <token>` header
- **Async Operations** — Mutating calls return a task ID; poll `/v1/tasks/{id}` for status
- **Resource Locks** — SDDC Manager locks resources during operations to prevent conflicts
- **JSON Payloads** — All request/response bodies use `Content-Type: application/json`
- **TLS Required** — All endpoints are HTTPS only; use `-k` or `--insecure` with curl for self-signed certs

---

# 2. Authentication & Token Management

## Endpoint: POST /v1/tokens

Authenticates a user and returns a JWT Bearer token for all subsequent API calls.

**Request:**

```
curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{
    "username": "administrator@vsphere.local",
    "password": "<password>"
  }'
```

**Response:**

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": {
    "id": "a1b2c3d4-..."
  }
}
```

## Token Details

| Property | Value |
|---|---|
| Access token lifetime | **60 minutes** |
| Refresh token lifetime | **24 hours** |
| Token type | JWT (JSON Web Token) |
| Required header | `Authorization: Bearer <accessToken>` |
| Supported users | `administrator@vsphere.local`, `admin@local` |

## Token Extraction (Shell Variable)

```
# One-liner to extract and store the token
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<password>"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# Verify it's set
echo "Token: ${TOKEN:0:20}..."
```

### Token Refresh

When the access token expires (HTTP 401), use the refresh token to get a new one without re-authenticating:

```
curl -sk -X PATCH https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"refreshToken":{"id":"<refresh-token-id>"}}'
```

### Token Scope & Permissions

| User | Scope | Notes |
|---|---|---|
| `administrator@vsphere.local` | Full admin — all endpoints | Default VCF admin account |
| `admin@local` | Full admin — all endpoints | SDDC Manager local admin |
| Custom SSO users | Role-based | Depends on assigned VCF role |

> ***Security Note:*** *Tokens are not invalidated server-side on logout in all versions. Always use HTTPS and protect token values. If a token is compromised, change the associated password immediately.*

---

# 3. System Health & Information

## GET /v1/system

Returns SDDC Manager system information including version and deployment type.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system | python3 -m json.tool
```

## GET /v1/system/health

Returns the overall health status of the VCF platform.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/health | python3 -m json.tool
```

**Response fields:**

| Field | Values | Meaning |
|---|---|---|
| `overallStatus` | `GREEN`, `YELLOW`, `RED` | Aggregate health |
| `componentHealth` | Array | Per-component health breakdown |

## GET /v1/system/notifications

Returns active system notifications (warnings, errors, info messages).

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/notifications | python3 -c \
  "import sys,json; [print(f'{n[\"severity\"]}: {n[\"message\"]}') for n in
json.load(sys.stdin).get('elements',[])]"
```

> ***Lab Note:*** *After a credential cascade failure, stale notifications may persist even after the fix. These clear after the operationsmanager service is restarted and the next health check cycle completes.*

---

# 4. Credentials Management

## Overview

The `/v1/credentials` endpoint manages all credentials stored inside SDDC Manager, including passwords for vCenter, NSX, ESXi hosts, and service accounts. This is a **high-privilege endpoint** that returns sensitive data.

## GET /v1/credentials

Lists all stored credentials.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/credentials | python3 -c "
import sys, json
data = json.load(sys.stdin)
for c in data.get('elements', []):
    print(f'{c[\"credentialType\"]:20s} {c[\"resource\"][\"resourceName\"]:30s}
{c[\"resource\"][\"resourceType\"]:15s} {c.get(\"accountType\",\"\")}')
"
```

## Credential Types

| credentialType | Description | Examples |
|---|---|---|
| `SSH` | SSH access credentials | ESXi root, SDDC Manager vcf |
| `API` | API/management credentials | NSX admin, vCenter admin |
| `SSO` | SSO domain credentials | administrator@vsphere.local |
| `SERVICE` | Service account credentials | Internal service accounts |

## Resource Types

| resourceType | Component |
|---|---|
| `ESXI` | ESXi host |
| `VCENTER` | vCenter Server |
| `NSX_MANAGER` | NSX Manager |
| `NSX_CONTROLLER` | NSX Controller |

| | |
|---|---|
| `SDDC_MANAGER` | SDDC Manager |
| `BACKUP` | Backup credentials |

### PUT /v1/credentials — Update Password

Updates a credential to match the current password on the target component (when SDDC Manager's stored password is out of sync).

```
curl -sk -X PUT https://sddc-manager.lab.local/v1/credentials \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "operationType": "UPDATE",
    "elements": [{
      "resourceName": "nsx-vip.lab.local",
      "resourceType": "NSX_MANAGER",
      "credentials": [{
        "credentialType": "API",
        "username": "admin",
        "password": "<current-password-on-nsx>"
      }]
    }]
  }'
```

***Returns a task ID** — poll* `/v1/tasks/{id}` *to monitor progress.*

### PUT /v1/credentials — Rotate Password

Generates a new password and pushes it to both SDDC Manager's store and the target component.

```
curl -sk -X PUT https://sddc-manager.lab.local/v1/credentials \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "operationType": "ROTATE",
    "elements": [{
      "resourceName": "nsx-vip.lab.local",
      "resourceType": "NSX_MANAGER",
      "credentials": [{
        "credentialType": "API",
```

```
        "username": "admin"
      }]
    }]
  }'
```

**WARNING — Credential Cascade Failure:** If a credential update or rotate fails mid-operation (e.g., NSX unreachable during boot storm), it triggers a cascade: NSX resource stuck in `ACTIVATING` / `ERROR` state in `platform.nsxt` table, stale locks in `platform.lock`, and unresolved tasks in `platform.task_metadata`. Each UI retry adds more stuck tasks. The API cannot cancel these tasks (`TA_TASK_CAN_NOT_BE_RETRIED`). Fix requires 6-step PostgreSQL repair — see the Troubleshooting Handbook Section 10.

---

# 5. NSX-T Cluster Inventory

## Overview

The `/v1/nsxt-clusters` endpoint retrieves information about NSX-T clusters managed by VCF, including health state, version, connection status, and cluster members.

## GET /v1/nsxt-clusters

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -m json.tool
```

### Key response fields:

| Field | Description | Values |
|---|---|---|
| `id` | NSX cluster UUID | UUID string |
| `status` | Connection status in SDDC Manager | `ACTIVE`, `ACTIVATING`, `ERROR` |
| `vipFqdn` | NSX VIP FQDN | `nsx-vip.lab.local` |
| `vip` | NSX VIP IP address | `192.168.1.70` |
| `nodes` | Array of NSX Manager nodes | Node FQDNs, IPs, status |

## Checking NSX Status (One-Liner)

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -c "
import sys, json
data = json.load(sys.stdin)
for c in data.get('elements', []):
    print(f'Cluster: {c[\"id\"]}')
    print(f'  Status: {c[\"status\"]}')
    print(f'  VIP: {c.get(\"vipFqdn\",\"?\")} ({c.get(\"vip\",\"?\")})')
    for n in c.get('nodes', []):
        print(f'  Node: {n.get(\"fqdn\",\"?\")} - {n.get(\"status\",\"?\")}')
"
```

> ***Lab Insight:*** *When* `status` *shows* `ACTIVATING` *or* `ERROR` *instead of* `ACTIVE`*, credential operations will fail. This is the first thing to check when credential remediation fails. If the status is stuck, direct PostgreSQL repair is required (see Section 7.2.6 of the Master Bible).*

---

# 6. Task Lifecycle & Monitoring

## Overview

SDDC Manager's API is **asynchronous** — most mutating operations (upgrades, credential rotations, deployments, domain creation) return a task ID. You must poll the task endpoint to determine success or failure.

## GET /v1/tasks

Lists all tasks, optionally filtered by status.

```
# All tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/tasks | python3 -m json.tool

# Only IN_PROGRESS tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" | python3 -c "
import sys, json
data = json.load(sys.stdin)
print(f'IN_PROGRESS tasks: {len(data.get(\"elements\",[]))}')
for t in data.get('elements', []):
    print(f'  {t[\"id\"][:8]}... {t[\"name\"]} ({t[\"status\"]})')
"

# Failed tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=FAILED"
```

## GET /v1/tasks/{id}

Returns detailed status of a specific task, including sub-tasks and error messages.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/tasks/<task-id> | python3 -c "
import sys, json
t = json.load(sys.stdin)
print(f'Task: {t[\"name\"]}')
print(f'Status: {t[\"status\"]}')
print(f'Created: {t.get(\"creationTimestamp\",\"?\")}')
for sub in t.get('subTasks', []):
```

```
    print(f'  Sub: {sub[\"name\"]} -> {sub[\"status\"]}')
    if sub.get('errors'):
        for e in sub['errors']:
            print(f'    ERROR: {e.get(\"message\",\"\")}')
"
```

### Task Statuses

| Status | Meaning |
|---|---|
| `IN_PROGRESS` | Task is currently executing |
| `SUCCESSFUL` | Task completed successfully |
| `FAILED` | Task failed — check sub-tasks for errors |
| `CANCELLED` | Task was cancelled (rare — API often refuses cancellation) |

### PATCH /v1/tasks/{id} — Cancel a Task

Attempts to cancel a running task. **This often fails for stuck tasks.**

```
curl -sk -X PATCH https://sddc-manager.lab.local/v1/tasks/<task-id> \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"status":"CANCELLED"}'
```

> ***Lab Discovery:*** *The API returns* `TA_TASK_CAN_NOT_BE_RETRIED` *for stuck tasks.* `DELETE /v1/tasks/{id}` *returns HTTP 500. When the API cannot cancel stuck tasks, direct PostgreSQL database repair is the only option — mark* `task_metadata.resolved = true` *in the* `platform` *database.*

### Task Polling Pattern

```
# Poll a task every 30 seconds until completion
TASK_ID="<task-id>"
while true; do
  STATUS=$(curl -sk -H "Authorization: Bearer $TOKEN" \
    https://sddc-manager.lab.local/v1/tasks/$TASK_ID \
    | python3 -c "import sys,json; print(json.load(sys.stdin)['status'])")
  echo "$(date +%H:%M:%S) - $STATUS"
  [ "$STATUS" = "SUCCESSFUL" ] || [ "$STATUS" = "FAILED" ] && break
```

```
  sleep 30
done
```

---

# 7. Resource Locks & Concurrency

## Overview

SDDC Manager uses exclusive resource locks to prevent conflicting operations on the same component. When a workflow starts, it acquires locks on the involved resources. If a workflow fails without releasing its locks, subsequent operations are blocked.

## GET /v1/resource-locks

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -m json.tool
```

**Response example:**

```
{
  "elements": [
    {
      "resource": "domain-1",
      "lockedBy": "task-12345",
      "lockType": "EXCLUSIVE"
    }
  ]
}
```

## Lock Types

| lockType | Meaning |
|---|---|
| `EXCLUSIVE` | Full exclusive lock — no other operations can touch this resource |
| `SHARED` | Read lock — other reads allowed, writes blocked |

## Lockable Resources

| Resource Type | Examples |
|---|---|
| Workload domains | Management domain, workload domains |
| Clusters | vSphere clusters |

| | |
|---|---|
| Hosts | Individual ESXi hosts |
| NSX objects | NSX Manager clusters, edge nodes |
| vCenter | vCenter Server instances |

## Diagnosing Lock Issues

```
# Check if any resources are locked
LOCKS=$(curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -c \
  "import sys,json; print(len(json.load(sys.stdin).get('elements',[])))")
echo "Active locks: $LOCKS"
```

***Key Insight:*** *Stale locks from failed operations are stored in the* `platform.lock` *PostgreSQL table. The API provides no way to force-release locks. When the API shows locks but no running tasks, direct database cleanup (* `DELETE FROM lock` *) is required.*

---

# 8. Hosts Management

## GET /v1/hosts

Lists all ESXi hosts managed by SDDC Manager.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/hosts | python3 -c "
import sys, json
for h in json.load(sys.stdin).get('elements', []):
    print(f'{h[\"fqdn\"]:30s} {h[\"status\"]:12s} {h.get(\"hardwareModel\",\"\")}')
"
```

## POST /v1/hosts — Commission a Host

Adds a new ESXi host to SDDC Manager's inventory.

```
curl -sk -X POST https://sddc-manager.lab.local/v1/hosts \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '[{
    "fqdn": "esxi05.lab.local",
    "username": "root",
    "password": "<password>",
    "networkPoolId": "<pool-id>",
    "storageType": "VSAN"
  }]'
```

## DELETE /v1/hosts/{id} — Decommission a Host

Removes an ESXi host from SDDC Manager's inventory.

```
curl -sk -X DELETE -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/hosts/<host-id>
```

## Host Statuses

| Status | Meaning |
| --- | --- |
| `ASSIGNED` | Host is assigned to a domain/cluster |
| `UNASSIGNED_USEABLE` | Available for assignment |
| `UNASSIGNED_UNUSEABLE` | Needs remediation before use |

---

# 9. Domains & Clusters

## GET /v1/domains

Lists all workload domains.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/domains | python3 -c "
import sys, json
for d in json.load(sys.stdin).get('elements', []):
    print(f'{d[\"name\"]:25s} {d[\"type\"]:15s} {d[\"status\"]}')
"
```

## GET /v1/clusters

Lists all vSphere clusters across all domains.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/clusters | python3 -c "
import sys, json
for c in json.load(sys.stdin).get('elements', []):
    print(f'{c[\"name\"]:25s} hosts={len(c.get(\"hosts\",[]))} {c[\"status\"]}')
"
```

## GET /v1/vcenters

Lists all vCenter Server instances managed by VCF.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/vcenters | python3 -m json.tool
```

---

# 10. Bundles & Lifecycle Updates

## GET /v1/bundles

Lists all available update bundles (downloaded or available for download).

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/bundles | python3 -c "
import sys, json
for b in json.load(sys.stdin).get('elements', []):
    print(f'{b.get(\"bundleType\",\"?\":15s} {b.get(\"version\",\"?\":12s}
{b.get(\"downloadStatus\",\"?\")}')
"
```

## GET /v1/upgradables

Lists components that have available upgrades.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/upgradables | python3 -m json.tool
```

## POST /v1/upgrades — Start an Upgrade

Initiates an upgrade for a specific component. Returns a task ID.

```
curl -sk -X POST https://sddc-manager.lab.local/v1/upgrades \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "bundleId": "<bundle-uuid>",
    "resourceType": "DOMAIN",
    "resourceUpgradeSpecs": [{
      "resourceId": "<domain-id>",
      "scheduledTimestamp": ""
    }]
  }'
```

## Upgrade Order (Always Follow This Sequence)

| Order | Component | Why |
|---|---|---|
| 1 | SDDC Manager | Orchestrates all other upgrades |
| 2 | vCenter Server | Required before ESXi upgrades |
| 3 | NSX Manager | Required before host network changes |
| 4 | ESXi Hosts | Rolling upgrade, one host at a time |
| 5 | vSAN | After all hosts are upgraded |
| 6 | VCF Operations | Last — depends on all infrastructure |

---

# 11. Notifications & Events

## GET /v1/system/notifications

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/notifications | python3 -c "
import sys, json
data = json.load(sys.stdin)
for n in data.get('elements', []):
    print(f'[{n[\"severity\"]}] {n[\"message\"][:100]}')
"
```

## Notification Severities

| Severity | Meaning |
|---|---|
| `INFO` | Informational — no action needed |
| `WARNING` | Potential issue — investigate |
| `ERROR` | Action required — operation failed or component unhealthy |
| `CRITICAL` | Immediate action — system at risk |

---

# 12. VCF Services & Depot

## GET /v1/services

Lists VCF services and their status.

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/services | python3 -m json.tool
```

## Network Pools

```
# List network pools
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/network-pools | python3 -m json.tool
```

## Personalities (vLCM Images)

```
# List available personalities
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/personalities | python3 -m json.tool
```

---

# 13. Complete Endpoint Reference

## Authentication

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/v1/tokens` | Authenticate and get Bearer token |
| `PATCH` | `/v1/tokens` | Refresh an expired access token |

## System

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/system` | System information and version |
| `GET` | `/v1/system/health` | Overall platform health |
| `GET` | `/v1/system/notifications` | Active notifications |
| `GET` | `/v1/system/dns-configuration` | DNS configuration |
| `GET` | `/v1/system/ntp-configuration` | NTP configuration |

## Infrastructure Inventory

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/hosts` | List all ESXi hosts |
| `POST` | `/v1/hosts` | Commission new host(s) |
| `DELETE` | `/v1/hosts/{id}` | Decommission a host |
| `GET` | `/v1/domains` | List all workload domains |
| `POST` | `/v1/domains` | Create a new workload domain |
| `DELETE` | `/v1/domains/{id}` | Delete a workload domain |
| `GET` | `/v1/clusters` | List all clusters |
| `POST` | `/v1/clusters` | Create a new cluster |
| `PATCH` | `/v1/clusters/{id}` | Expand/shrink a cluster |
| `GET` | `/v1/vcenters` | List all vCenter instances |
| `GET` | `/v1/nsxt-clusters` | List all NSX-T clusters |

## Credentials

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/credentials` | List all stored credentials |
| `PUT` | `/v1/credentials` | Update, rotate, or remediate credentials |
| `GET` | `/v1/credentials/{id}` | Get a specific credential |

## Tasks & Locks

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/tasks` | List all tasks (filter by `?status=` ) |
| `GET` | `/v1/tasks/{id}` | Get task details with sub-tasks |
| `PATCH` | `/v1/tasks/{id}` | Attempt to cancel a task |
| `GET` | `/v1/resource-locks` | List active resource locks |

## Lifecycle & Bundles

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/bundles` | List available update bundles |
| `POST` | `/v1/bundles` | Download a bundle |
| `GET` | `/v1/upgradables` | List upgradable components |
| `POST` | `/v1/upgrades` | Start an upgrade operation |
| `GET` | `/v1/upgrades` | List upgrade history |

## Networking & Pools

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/network-pools` | List network pools |
| `POST` | `/v1/network-pools` | Create a network pool |
| `GET` | `/v1/sddc-managers` | SDDC Manager information |
| `GET` | `/v1/personalities` | List vLCM personality images |

## Services

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/v1/services` | List VCF services |

---

# 14. Python Automation Scripts

## Full API Client Script

```python
#!/usr/bin/env python3
"""VCF SDDC Manager API client — query all key endpoints."""
import requests, json, sys, io, urllib3

urllib3.disable_warnings()
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')

SDDC = "https://sddc-manager.lab.local"
USER = "administrator@vsphere.local"
PASS = "Success01!0909!!"

# Authenticate
resp = requests.post(f"{SDDC}/v1/tokens",
    json={"username": USER, "password": PASS}, verify=False)
token = resp.json()["accessToken"]
headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}

def get(endpoint, label=""):
    url = f"{SDDC}{endpoint}"
    r = requests.get(url, headers=headers, verify=False)
    print(f"\n{'='*60}")
    print(f"GET {endpoint}  [{r.status_code}]  {label}")
    print('='*60)
    if r.status_code == 200:
        print(json.dumps(r.json(), indent=2)[:2000])
    else:
        print(f"Error: {r.text[:500]}")

# Query all key endpoints
get("/v1/system", "System Info")
get("/v1/system/health", "Platform Health")
get("/v1/credentials", "Credential Inventory")
get("/v1/nsxt-clusters", "NSX-T Clusters")
get("/v1/resource-locks", "Resource Locks")
get("/v1/tasks?status=IN_PROGRESS", "In-Progress Tasks")
get("/v1/hosts", "ESXi Hosts")
get("/v1/domains", "Workload Domains")
```

```
get("/v1/clusters", "vSphere Clusters")
get("/v1/system/notifications", "Notifications")
```

### Credential Status Checker

```
#!/usr/bin/env python3
"""Check credential status for all VCF components."""
import requests, json, sys, io, urllib3

urllib3.disable_warnings()
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')

SDDC = "https://sddc-manager.lab.local"
USER = "administrator@vsphere.local"
PASS = "Success01!0909!!"

resp = requests.post(f"{SDDC}/v1/tokens",
    json={"username": USER, "password": PASS}, verify=False)
token = resp.json()["accessToken"]
headers = {"Authorization": f"Bearer {token}"}

# Get credentials
r = requests.get(f"{SDDC}/v1/credentials", headers=headers, verify=False)
creds = r.json().get("elements", [])

print(f"{'Type':<12} {'Resource':<30} {'ResourceType':<18} {'Account':<25} {'Status'}")
print("-" * 100)
for c in creds:
    print(f'{c["credentialType"]:<12} '
          f'{c["resource"]["resourceName"]:<30} '
          f'{c["resource"]["resourceType"]:<18} '
          f'{c.get("username",""):<25} '
          f'{c.get("status","?")}')
```

### Task Monitor

```
#!/usr/bin/env python3
"""Monitor a VCF task until completion."""
import requests, json, sys, io, time, urllib3
```

```
urllib3.disable_warnings()
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace')

SDDC = "https://sddc-manager.lab.local"
USER = "administrator@vsphere.local"
PASS = "Success01!0909!!"
TASK_ID = sys.argv[1] if len(sys.argv) > 1 else input("Task ID: ")

resp = requests.post(f"{SDDC}/v1/tokens",
    json={"username": USER, "password": PASS}, verify=False)
token = resp.json()["accessToken"]
headers = {"Authorization": f"Bearer {token}"}

while True:
    r = requests.get(f"{SDDC}/v1/tasks/{TASK_ID}", headers=headers, verify=False)
    task = r.json()
    status = task["status"]
    name = task.get("name", "Unknown")
    print(f"[{time.strftime('%H:%M:%S')}] {name}: {status}")

    if status in ("SUCCESSFUL", "FAILED", "CANCELLED"):
        if status == "FAILED":
            for sub in task.get("subTasks", []):
                if sub.get("errors"):
                    for e in sub["errors"]:
                        print(f"  ERROR: {e.get('message','')}")
        break
    time.sleep(30)
```

## 15. Postman Collection

Save the following as `VCF_API_Collection.json` and import into Postman:

```
{
  "info": {
    "name": "VMware Cloud Foundation (VCF) - SDDC Manager API",
    "description": "REST API collection for VCF /v1/ endpoints",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "1 - Authenticate (Get Token)",
      "request": {
        "method": "POST",
        "header": [{"key": "Content-Type", "value": "application/json"}],
        "body": {
          "mode": "raw",
          "raw": "{\n  \"username\": \"administrator@vsphere.local\",\n  \"password\":
\"YOUR_PASSWORD\"\n}"
        },
        "url": "https://sddc-manager.lab.local/v1/tokens"
      }
    },
    {
      "name": "2 - System Health",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/system/health"
      }
    },
    {
      "name": "3 - Credentials Inventory",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/credentials"
      }
    },
    {
      "name": "4 - NSX-T Clusters",
```

```
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/nsxt-clusters"
      }
    },
    {
      "name": "5 - All Tasks",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/tasks"
      }
    },
    {
      "name": "6 - IN_PROGRESS Tasks",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS"
      }
    },
    {
      "name": "7 - Resource Locks",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/resource-locks"
      }
    },
    {
      "name": "8 - Hosts",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/hosts"
      }
    },
    {
      "name": "9 - Domains",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/domains"
      }
```

```
    },
    {
      "name": "10 - Clusters",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/clusters"
      }
    },
    {
      "name": "11 - Notifications",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/system/notifications"
      }
    },
    {
      "name": "12 - Bundles",
      "request": {
        "method": "GET",
        "header": [{"key": "Authorization", "value": "Bearer {{token}}"}],
        "url": "https://sddc-manager.lab.local/v1/bundles"
      }
    }
  ],
  "variable": [
    {"key": "token", "value": ""},
    {"key": "task_id", "value": ""}
  ]
}
```

**How to use:**

1. Import JSON into Postman
2. Run **"1 - Authenticate"** — copy `accessToken` from response
3. Set the `token` variable in Postman's Collection Variables
4. Call all other endpoints

---

# 16. Troubleshooting API Issues

## Common HTTP Status Codes

| Code | Meaning | Action |
|---|---|---|
| `200` | Success | Request completed |
| `201` | Created | Resource created, check for task ID |
| `202` | Accepted | Async task started, poll `/v1/tasks/{id}` |
| `400` | Bad Request | Check JSON payload syntax |
| `401` | Unauthorized | Token expired — re-authenticate |
| `403` | Forbidden | Insufficient permissions for this endpoint |
| `404` | Not Found | Resource ID doesn't exist |
| `409` | Conflict | Resource is locked by another operation |
| `500` | Internal Server Error | SDDC Manager issue — check logs |

## Common API Errors

| Error Message | Root Cause | Fix |
|---|---|---|
| `TA_TASK_CAN_NOT_BE_RETRIED` | Stuck task cannot be cancelled via API | Database repair: `UPDATE task_metadata SET resolved = true` |
| `Unable to acquire resource level lock(s)` | Stale locks from failed operation | Database repair: `DELETE FROM lock` in platform DB |
| `Resources [X] are not in ACTIVE state` | NSX cluster stuck in ACTIVATING/ERROR | Database repair: `UPDATE nsxt SET status = 'ACTIVE'` |
| `Connection refused` | SDDC Manager services not running | SSH in and run `systemctl restart vcf-services` |
| `SSL certificate problem` | Self-signed cert not trusted | Use `-k` flag with curl, or import cert |

## Debugging API Calls

```
# Verbose curl output (shows headers and TLS handshake)
curl -svk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/health 2>&1
```

```
# Check if SDDC Manager is reachable
curl -sk -o /dev/null -w "%{http_code}" \
  https://sddc-manager.lab.local/v1/system

# Test token validity
curl -sk -o /dev/null -w "%{http_code}" \
  -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system
# 200 = token valid, 401 = token expired
```

## SDDC Manager API Log Locations

```
# SSH to SDDC Manager as vcf, su - to root
/var/log/vmware/vcf/operationsmanager/operationsmanager.log  # Credential ops
/var/log/vmware/vcf/domainmanager/domainmanager.log          # Domain/cluster ops
/var/log/vmware/vcf/lcm/lcm.log                             # Lifecycle/upgrade ops
/var/log/vmware/vcf/commonsvcs/commonsvcs.log                # Auth/token issues
```

---

# 17. Lab-Tested Workflows

## Workflow 1: Full Health Check

```
# 1. Authenticate
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<pass>"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# 2. System health
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/health

# 3. NSX status
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters

# 4. Check for stuck tasks
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS"

# 5. Check for stale locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks

# 6. Check notifications
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/system/notifications
```

## Workflow 2: Credential Update for NSX

```
# 1. Authenticate
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<pass>"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# 2. Verify NSX is ACTIVE first
```

```
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -c \
  "import sys,json; [print(c['status']) for c in json.load(sys.stdin).get('elements',
[])]"
# MUST show ACTIVE before proceeding

# 3. Verify no stale locks
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks

# 4. Submit credential update
TASK=$(curl -sk -X PUT https://sddc-manager.lab.local/v1/credentials \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "operationType": "UPDATE",
    "elements": [{
      "resourceName": "nsx-vip.lab.local",
      "resourceType": "NSX_MANAGER",
      "credentials": [{
        "credentialType": "API",
        "username": "admin",
        "password": "<current-nsx-password>"
      }]
    }]
  }' | python3 -c "import sys,json; print(json.load(sys.stdin).get('id','?'))")

echo "Task: $TASK"

# 5. Poll task status
while true; do
  S=$(curl -sk -H "Authorization: Bearer $TOKEN" \
    https://sddc-manager.lab.local/v1/tasks/$TASK \
    | python3 -c "import sys,json; print(json.load(sys.stdin)['status'])")
  echo "$(date +%H:%M:%S) $S"
  [ "$S" = "SUCCESSFUL" ] || [ "$S" = "FAILED" ] && break
  sleep 15
done
```

### Workflow 3: Diagnose Credential Cascade Failure

```
# 1. Authenticate
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<pass>"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# 2. Check NSX status (ACTIVATING/ERROR = problem)
echo "=== NSX Status ==="
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/nsxt-clusters | python3 -c \
  "import sys,json; [print(f'  {c[\"id\"][:8]}... status={c[\"status\"]}') for c in
json.load(sys.stdin).get('elements',[])]"

# 3. Check resource locks (stale locks = problem)
echo "=== Resource Locks ==="
curl -sk -H "Authorization: Bearer $TOKEN" \
  https://sddc-manager.lab.local/v1/resource-locks | python3 -c \
  "import sys,json; d=json.load(sys.stdin).get('elements',[]); print(f'  Locks:
{len(d)}'); [print(f'  {l}') for l in d[:5]]"

# 4. Check stuck tasks (dozens = cascade failure)
echo "=== Stuck Tasks ==="
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" | python3 -c \
  "import sys,json; d=json.load(sys.stdin).get('elements',[]); print(f'  IN_PROGRESS:
{len(d)}')"

# 5. Check latest failed tasks
echo "=== Recent Failed Tasks ==="
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://sddc-manager.lab.local/v1/tasks?status=FAILED" | python3 -c "
import sys, json
tasks = json.load(sys.stdin).get('elements', [])[:3]
for t in tasks:
    print(f'  {t[\"name\"]}: {t[\"status\"]}')
    for sub in t.get('subTasks', []):
        if sub.get('errors'):
            for e in sub['errors']:
                print(f'    ERROR: {e.get(\"message\",\"\")[:100]}')
"

# 6. Verify actual NSX health (bypass SDDC Manager)
echo "=== Direct NSX VIP Check ==="
```

```
curl -sk -u admin:'<pass>' --connect-timeout 10 \
  https://nsx-vip.lab.local/api/v1/cluster/status | python3 -c \
  "import sys,json; d=json.load(sys.stdin); print(f'  Overall:
{d.get(\"detailed_cluster_status\",{}).get(\"overall_status\",\"?\")}')" 2>/dev/null ||
echo "  VIP unreachable"
```

---

# 18. References

| Resource | URL |
|---|---|
| SDDC Manager API Reference | developer.broadcom.com/xapis/sddc-manager-api/latest/ |
| VCF API Reference Guide | developer.broadcom.com/xapis/vmware-cloud-foundation-api/latest/ |
| OpenAPI for SDDC Manager | techdocs.broadcom.com |
| VCF Troubleshooting Handbook | `VCF_Troubleshooting_Handbook.md` — Section 10 (Credential Cascade) |
| VCF Master Bible | `VCF9-Master-Bible.md` — Section 7.2.6 (Database Repair), Section 8.7 (API Quick Reference) |
| VCF Command Reference | `VCF-Command-Reference.md` — Sections 24-25 (SDDC Manager API, PostgreSQL) |
| Diagnostic Scripts | `diagnostic-scripts/` — `quick_status.py`, `nsx_cred_update.py`, `full_remediate_fix.py` |

---

### Document Information

| Field | Value |
|---|---|
| Document Title | VCF 9.0.1 SDDC Manager REST API Handbook |
| Version | 1.0 |
| Last Updated | February 2026 |
| Source Material | SDDC API Rest points.docx, Broadcom API Reference, Lab testing |
| Environment | Dell Precision 7920, VMware Workstation Nested Lab |
| SDDC Manager | 9.0.1.0 build 24962180 |

*This handbook documents the SDDC Manager REST API as tested in a VCF 9.0.1 nested lab environment. API behavior may vary in production deployments. Always consult the official Broadcom API reference for the latest endpoint specifications.*

*(c) 2026 Virtual Control LLC. All rights reserved.*